

AhamX Bodhi

Where I Becomes Infinite

Embeddings

Module 2.2: Transformer Architecture Fundamentals

Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- What an embedding is and why it is needed
- How tokens map to dense vector representations
- Token embeddings vs. positional encodings
- How the embedding matrix is learned and used

Concept at a Glance

- **Duration:** 9 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate
- **Module:** 2.2 Transformer Architecture Fundamentals
- **Prerequisite:** Tokenization, Transformer Overview



Learning Objectives

By the End of This Session You Will Be Able To

1. **Define** an embedding and explain why transformers require continuous vector inputs (B1)
2. **Describe** how the token embedding matrix maps vocabulary indices to dense vectors (B2)
3. **Explain** the role of the embedding dimension d_{model} in transformer design (B2)
4. **Distinguish** token embeddings from positional encodings and understand their combination (B2)
5. **Relate** embedding matrix parameter count to total model size and pre-training cost (B2)



Prerequisites and Context

Prerequisites

- What a Transformer is (Module 2.2)
- What tokenization does and what a vocabulary is
- Familiarity with vectors and matrices
- Basic understanding of neural network parameters

This Concept Enables

- Self-Attention Mechanism (Module 2.2)
- Positional Encoding deep dive
- Fine-tuning and embedding adaptation (Module 7)
- RAG: semantic search via embedding similarity
- LoRA: low-rank updates to embedding layers



The Core Problem --- Computers Cannot Read Words

Why Embeddings Are Necessary

- Neural networks operate on real-valued numbers, not text strings
- After tokenization, each token is represented as an integer index (e.g., token 4921)
- A raw integer cannot encode semantic relationships between words
- The integer 4921 is not meaningfully "close to" 4922 just because the numbers are adjacent

What We Need

- A learned mapping from integer token ID $\rightarrow$ dense continuous vector
- The vector should capture semantic and syntactic properties
- Similar words should produce similar (nearby) vectors



What is an Embedding?

Formal Definition

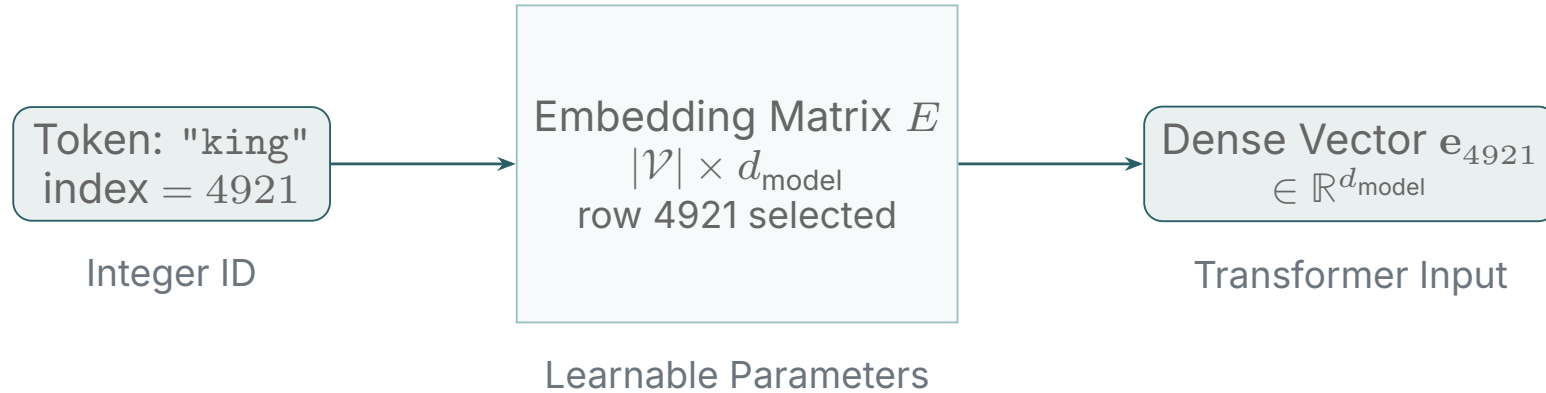
- An **embedding** is a learned function that maps a discrete symbol (token) to a dense, fixed-dimensional real-valued vector
- The **embedding matrix** $E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ has one row per vocabulary token
- Looking up token i returns the i -th row: $\mathbf{e}_i = E[i, :] \in \mathbb{R}^{d_{\text{model}}}$
- The values in E are **learned parameters** — trained jointly with all other weights

The embedding layer is the very first transformation inside a transformer: raw token IDs become continuous vectors that every subsequent layer can process.





The Embedding Matrix --- Lookup Operation



Key Insight

- The lookup is **differentiable**: gradients flow back through E during training
- E is updated by backpropagation exactly like any other weight matrix



The Embedding Dimension d_{model}

Why Dimension Matters

- d_{model} is the width of the embedding vector and the hidden size of the entire transformer
- Larger d_{model} allows richer representations but increases all layer sizes proportionally
- Every attention head, feed-forward sublayer, and output projection uses d_{model}
- Changing d_{model} changes the *entire* model — it is the single most important architectural hyperparameter

Typical Values in Production Models

- **GPT-2 Small:** $d_{\text{model}} = 768$
- **GPT-3 / GPT-4 class:** $d_{\text{model}} = 12,288$ (GPT-3 175B)
- **Llama 3 8B:** $d_{\text{model}} = 4,096$
- **TinyLlama 1.1B:** $d_{\text{model}} = 2,048$



Parameter Count of the Embedding Matrix

Formula

- Number of parameters in the embedding layer: $N_{\text{emb}} = |\mathcal{V}| \times d_{\text{model}}$
- GPT-2 Small: $50,257 \times 768 \approx 38.6\text{M}$ parameters
- Llama 3 8B: $128,256 \times 4,096 \approx 525\text{M}$ parameters (6.6% of total)
- The embedding matrix can be a **significant fraction** of total model parameters

Weight Tying

- Many architectures **tie weights**: the input embedding matrix E is reused as the output projection ("lm head") matrix
- This halves the parameter count while improving training stability
- GPT-2 uses weight tying; Llama 3 does *not*



Geometric Interpretation of Embeddings

Embeddings as Points in Vector Space

- Each token is a **point** in $\mathbb{R}^{d_{\text{model}}}$
- Semantically similar tokens cluster together after training
- The celebrated **analogy relationship** holds approximately:
$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}$$
- Distance metrics (cosine similarity, Euclidean) quantify token relatedness

Implications for Downstream Tasks

- Semantic search uses cosine similarity between embeddings
- Nearest-neighbour retrieval powers RAG pipelines
- Embedding visualisation (t-SNE, UMAP) reveals learned topic clusters
- Transfer learning works because embeddings generalise across tasks



Token Embeddings vs. Positional Encodings

Token Embedding e_i

- Encodes **what** the token means (identity, semantics)
- Looked up from matrix E by token ID
- **Shared** across all positions — same token, same embedding
- Learned during pre-training

Positional Encoding p_i

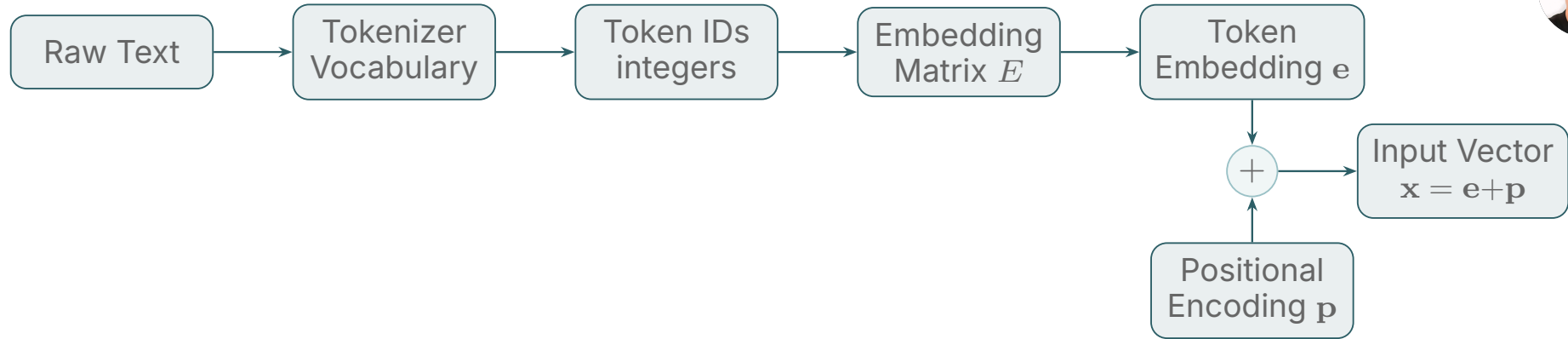
- Encodes **where** the token sits in the sequence
- Added to (or fused with) token embedding
- Unique per position; position-aware
- Fixed (sinusoidal) or learned (GPT-2 style)

Combined Input to Transformer

- Final input vector: $x_i = e_i + p_i$
- This combined vector carries both *identity* and *order* information



Full Token-to-Vector Pipeline



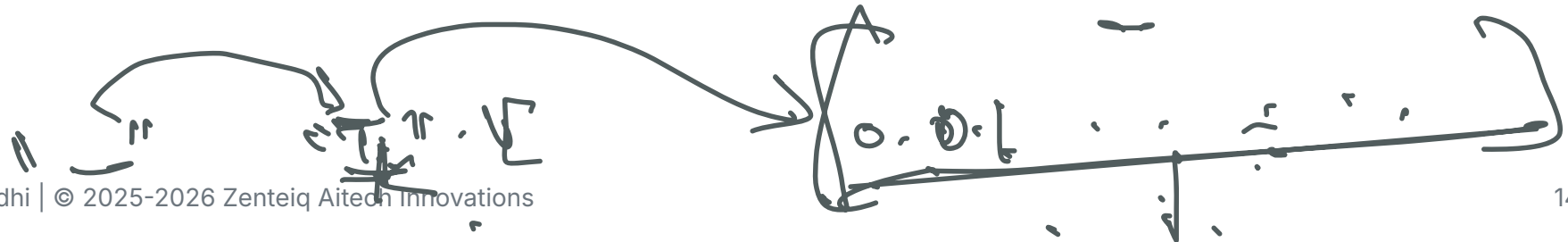
Pipeline Summary

- Token IDs are looked up in E ; positional encodings are added element-wise
- The result x is fed to the first multi-head attention layer



Types of Positional Encoding

Method		How It Works	Used In
Absolute Soidal	Sinu-	Fixed $\sin / \cos$ functions of position	Original (2017) Transformer
Learned Absolute		Trainable vector per position index	GPT-2, BERT
Relative (Shaw)		Bias added inside attention to relative offsets	Transformer-XL
RoPE		Rotates query/key vectors by position angle	Llama, Mistral, Gemma
ALiBi		Linear bias on attention scores; no extra params	MPT, BLOOM



Types of Positional Encoding



Trend

- Modern open models (Llama 2/3, Mistral) use **RoPE** for better length generalisation
- RoPE encodes position inside the attention computation rather than by adding to embeddings

GenAI Application --- Semantic Search and RAG



How Embeddings Power Retrieval-Augmented Generation

- Documents are chunked and each chunk is encoded to an embedding vector via an encoder model
- Embeddings are stored in a **vector database** (FAISS, Pinecone, Weaviate, ChromaDB)
- At query time, the user question is embedded and the nearest-neighbour chunks are retrieved
- Retrieved chunks are injected into the LLM context as grounding evidence

Production Systems

- **OpenAI text-embedding-3-large** ($d = 3,072$): general-purpose semantic search
- **Cohere embed-v3**: multilingual retrieval for enterprise document systems
- **BGE / E5 models**: open-weight bi-encoders used in self-hosted RAG stacks
- **LangChain / LlamaIndex**: orchestration libraries wrapping embedding + retrieval



GenAI Application --- Fine-Tuning and Embedding Adaptation

Why Embedding Layers Matter in Fine-Tuning

- When a domain uses specialised vocabulary (medical, legal, code), pre-trained embeddings may not capture nuanced meanings
- Full fine-tuning updates E alongside all transformer weights
- **Embedding-only fine-tuning** freezes other weights and only updates E — very efficient for vocabulary adaptation
- Domain-specific tokenisers may extend the vocabulary, requiring new embedding rows



GenAI Application --- Fine-Tuning and Embedding Adaptation

LoRA and Embeddings

- Standard LoRA does *not* apply low-rank updates to E (it is already low-rank in practice)
- **LoftQ** and **LLaMA-Adapter** show that embedding-layer adaptation aids task transfer
- Extended vocabularies (e.g., code tokens in CodeLlama) are initialised from neighbour embeddings



GenAI Application --- Multimodal Embeddings

Extending Embeddings Beyond Text

- Vision-language models (CLIP, LLaVA, GPT-4V) project image patches into the same d_{model} -dimensional space as text tokens
- A **vision encoder** (ViT) produces patch embeddings; a linear projection aligns them with the text embedding space
- The transformer then processes a sequence of mixed text and image embeddings uniformly
- Audio tokens (Whisper), video frames, and molecule graphs follow the same pattern

Examples in Production

- **GPT-4o**: unified token stream of text, image, and audio embeddings
- **LLaVA-1.6**: open-weight VLM using Llama 3 backbone with ViT projector
- **CLIP**: joint image-text embedding space for zero-shot image classification



GenAI Application --- Sentence and Document Embeddings

From Token Embeddings to Sentence Embeddings

- Transformer token embeddings are contextual: the same token gets different vectors depending on its surrounding context (unlike static Word2Vec)
- To represent an entire sentence, a **pooling** operation aggregates token embeddings: mean pooling, max pooling, or the [CLS] token representation
- Sentence-BERT (SBERT) adds a contrastive loss to learn semantically meaningful sentence-level representations

Practical Uses

- Clustering customer support tickets by semantic topic
- Duplicate question detection in Q&A platforms
- Cross-lingual document matching without translation
- Recommendation systems using user history embeddings



Static vs. Contextual Embeddings

Static Embeddings (Word2Vec, GloVe)

- One fixed vector per word, regardless of context
- "bank" always maps to the same vector
- Fast and lightweight; still used in some NLP tasks
- Cannot distinguish polysemy (multiple meanings)

Contextual Embeddings (BERT, GPT)

- Vector for each token *depends on surrounding context*
- "bank" has different vectors in "river bank" vs. "savings bank"
- Richer representations; basis of modern NLP
- Computationally heavier; requires forward pass

Key Point

- Transformer token embeddings E are *static* (lookup table); context is added by the **attention layers** that follow — both components are essential



Measuring Similarity in Embedding Space

Cosine Similarity

- The most common metric for comparing embedding vectors:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- Range $[-1, 1]$: 1 = identical direction, 0 = orthogonal, -1 = opposite
- Normalised inner product — insensitive to vector magnitude
- Preferred over Euclidean distance for high-dimensional semantic spaces

Why Cosine, Not Euclidean?

- In high dimensions, all points are roughly equidistant (curse of dimensionality)
- Cosine captures *orientation* (meaning direction), not magnitude
- FAISS vector databases support both; cosine is standard in text retrieval



Skills Enabled by This Concept

Conceptual Skills

- Explain why transformers need continuous vector inputs
- Read model configs and interpret d_{model} , vocab size, parameter count
- Distinguish token, sentence, and document-level embeddings
- Evaluate positional encoding choices for long-context tasks

Practical Skills

- Use `model.embed_tokens` in HuggingFace to inspect the embedding matrix
- Build a semantic search pipeline with FAISS and sentence-transformers
- Compute cosine similarity for nearest-neighbour retrieval
- Estimate embedding layer parameters for budget planning



Professional Roles That Rely on Embedding Knowledge

Role	How Embeddings Are Used
ML Engineer	Configures embedding layers; tunes d_{model} for speed/quality trade-off
NLP Researcher	Probes embedding geometry; studies polysemy and semantic shift
RAG Architect	Selects embedding model; manages vector database indexing strategy
Fine-Tuning Engineer	Decides whether to freeze or update E during domain adaptation
AI Product Manager	Chooses embedding API (cost, latency, dimension, multilingual support)



Knowledge Graph --- Where This Concept Sits

Concepts That Lead Here (Prerequisites)

- **AI-TR-FN-TH-000001** — What is a Transformer? (Module 2.2 entry point)
- **AI-TK-FN-TH-000001** — What is Tokenization? (vocabulary and token IDs)
- **AI-TR-FN-TH-000004** — Positional Encoding (combined with embeddings)
- **AI-LM-FN-TH-000004** — Model Sizing and Scaling Laws (parameter count context)
- **AI-LM-FN-TH-000005** — Foundation Models Overview (architectural motivation)



What This Concept Unlocks --- Part 1

Immediate Next Concepts in Module 2.2

- **AI-TR-FN-TH-000002** — Self-Attention Mechanism (operates on embedding vectors)
- **AI-TR-FN-TH-000003** — Multi-Head Attention (extends self-attention)
- **AI-TR-FN-TH-000006** — Feed-Forward Sublayer (processes embedded representations)
- **AI-TR-FN-TH-000007** — Layer Normalisation (applied to embedding + attention output)



What This Concept Unlocks --- Part 2

Downstream Concepts Across the Programme

- **AI-RA-IN-TH-000001** — RAG Architecture Overview (semantic retrieval)
- **AI-FT-IN-TH-000006** — LoRA (parameter-efficient updates near embedding layers)
- **AI-PT-IN-TH-000001** — Pre-Training Overview (embedding matrix initialisation)
- **AI-MM-IN-TH-000001** — Multimodal Systems (projecting non-text modalities)
- **AI-OP-IN-TH-000002** — Quantisation and Compression (embedding quantisation)



Common Misconceptions

Watch Out for These

- **"Embeddings encode meaning by themselves"** — *False*. Raw embeddings encode token identity; contextual meaning emerges from the attention layers stacked on top.
- **"Larger embedding dimension always means better quality"** — *False*. Diminishing returns appear quickly; Llama 3 8B ($d=4096$) outperforms many models with $d=12288$.
- **"Embeddings are fixed after pre-training"** — *False*. They are updated by fine-tuning unless explicitly frozen.
- **"Word2Vec and transformer embeddings are the same thing"** — *False*. Word2Vec is static; transformer embeddings are contextual.

Summary



Embeddings — Key Takeaways

1. An **embedding** maps a discrete token ID to a dense vector $\mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$ via a learned matrix E
2. The embedding dimension d_{model} governs the width of the entire transformer
3. Token embeddings encode *identity*; positional encodings encode *order*; both are summed to form the transformer input $\mathbf{x} = \mathbf{e} + \mathbf{p}$
4. The embedding matrix E is a **learnable parameter** updated by backpropagation
5. Embeddings are the foundation of semantic search, RAG, fine-tuning, and multimodal AI

Next Steps



Immediately After This Session

- Inspect an embedding matrix in HuggingFace:
`model.embed_tokens.weight.shape`
- Run a semantic search demo with `sentence-transformers` library
- Compute cosine similarity between token embeddings for synonym pairs

Coming Up in Module 2.2

- Self-Attention Mechanism (AI-TR-FN-TH-000002)
- Multi-Head Attention (AI-TR-FN-TH-000003)
- Feed-Forward Sublayer (AI-TR-FN-TH-000006)
- Layer Normalisation (AI-TR-FN-TH-000007)

Thank You

Embeddings | Module 2.2: Transformer Architecture
Fundamentals